# HyperBalance: Hypergraph-based Topology Optimization for Reliable Multi-Agent Communication

**Anonymous Author(s)**

## Abstract

Communication topology design has become central to the effectiveness of large language model based multi-agent systems. Recent advances introduce hypergraph representations that connect multiple agents within unified hyperedges, enabling one-step synchronization for group collaboration. However, existing hypergraph methods assume that information flow within hyperedges is always beneficial. This assumption overlooks a critical risk. The same synchronization mechanism can accelerate not only valuable insight sharing but also erroneous output propagation. In this paper, we extend causal analysis to hypergraph structures and reveal that hyperedge scale governs a trade-off between error suppression and insight retention. Based on this finding, we propose **HyperBalance**, a dual-view hypergraph topology learning framework. HyperBalance employs sparse-view and dense-view encoders to separately model error suppression and insight propagation. A query-aware fusion module adaptively combines these views to generate task-specific topologies. Experiments on six benchmarks show that HyperBalance achieves 91.77% average accuracy, outperforming prior methods by 2.99%, and reduces token consumption by 25.33% compared to dense baselines.

## 1 Introduction

Large language model powered agents have demonstrated strong capabilities in reasoning, code generation, and complex decision making [Wei *et al.*, 2022a; Yao *et al.*, 2023a; Brown *et al.*, 2020]. Organizing multiple agents into collaborative systems can amplify their collective intelligence beyond individual limits [Du *et al.*, 2024a; Chen *et al.*, 2024b]. Central to such collaboration is communication topology. It defines information flow pathways among agents and shapes coordination patterns and reasoning dynamics. Effective topologies enable agents to integrate diverse perspectives, synchronize efficiently, and converge on accurate solutions. As multi-agent systems tackle more challenging applications, topology design becomes increasingly important for reliable collaboration.

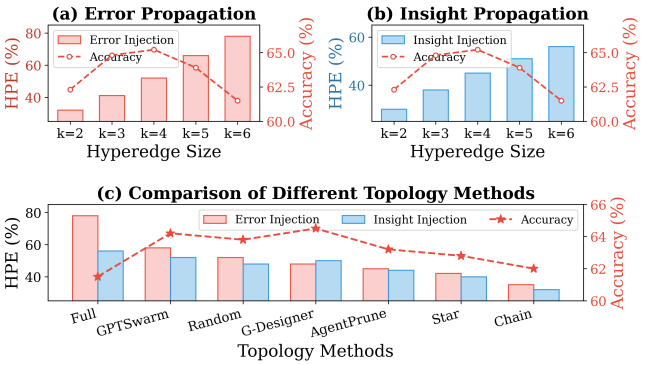


Figure 1: **Analysis of hyperedge propagation effects.** (a) Error propagation grows steeply with hyperedge size. (b) Insight propagation exhibits slower growth. Task accuracy peaks at intermediate sizes in both cases. (c) Comparison of topology methods reveals varied trade-offs between error suppression and insight sharing across existing approaches.

Existing research has explored diverse approaches to topology design. Static methods employ predefined structures such as chains, stars, trees, and fully connected graphs [Qian *et al.*, 2023; Hong *et al.*, 2024]. These provide consistent execution patterns suited for specific scenarios. Learning-based approaches shift toward task-adaptive designs. AgentPrune [Zhang *et al.*, 2025a] learns sparse masks to remove redundant connections. AgentDropout [Wang *et al.*, 2025] introduces stochastic pruning on nodes and edges. G-Designer [Zhang *et al.*, 2025b] employs graph generators to produce query-dependent topologies. These graph-based methods model agent relationships as pairwise connections and rely on multi-hop message passing for group coordination [Zhuge *et al.*, 2024; Hu *et al.*, 2024]. Hypergraph representations take a different approach. Hyperedges directly connect multiple agents within the same collaboration group and enable one-step information aggregation. This structure captures group-level interactions more naturally than pairwise edges. Existing hypergraph methods focus on improving synchronization efficiency [Zhu *et al.*, 2024]. They implicitly assume that information flow within hyperedges always benefits the system.

The assumption of universally beneficial information flow overlooks a critical concern. Hypergraph propagation differs substantially from pairwise graphs. In pairwise graphs, in-

formation travels through sequential hops. Each intermediate agent receives inputs from predecessors and generates outputs for successors. This sequential process provides natural opportunities for error correction. When one agent produces incorrect outputs, downstream agents may still reach correct conclusions by aggregating from multiple sources. Hypergraphs eliminate this sequential filtering. All agents in a hyperedge receive the same aggregated information simultaneously. They process shared input in parallel and contribute outputs to the next aggregation round. This synchronous mechanism offers efficiency benefits for collaborative reasoning. It also removes intermediate checkpoints that could catch errors. When an agent in a hyperedge produces erroneous outputs, all other members receive this error without opportunity for gradual correction. Prior work on pairwise graphs has identified a trade-off between error suppression and insight propagation. This trade-off remains unexplored in hypergraph settings.

We conduct systematic analysis to understand how hypergraph topologies affect information propagation. We extend causal intervention methods to hypergraph structures. We introduce Hyperedge Propagation Effect to quantify how individual agent outputs influence collective decisions. We apply two types of interventions across varying hyperedge sizes. Error injection forces selected agents to produce incorrect outputs. Insight injection provides selected agents with correct answers. Our experiments reveal an asymmetric pattern. Both error propagation and insight propagation increase with hyperedge size. Error propagation exhibits a steeper growth curve. As hyperedges grow larger, error amplification exceeds insight enhancement. Figure 1 illustrates this finding. The left panel shows error propagation growing faster than insight propagation. The right panel shows task accuracy peaking at intermediate hyperedge sizes. This asymmetry carries important implications for topology design. The optimal hyperedge configuration depends on agent reliability distribution in each task. Different agent combinations may require different collaboration intensities within the same task.

This finding motivates shifting from global uniformity to combination-level adaptivity. Existing methods apply uniform sparsity constraints across all agent groups. They do not distinguish collaboration intensity among different combinations. We propose HyperBalance to address this gap. The framework employs two complementary hypergraph convolutional branches. Error suppression favors sparse connectivity to limit propagation paths. Insight diffusion benefits from dense connectivity to enable broad sharing. Each branch specializes in one objective. A query-aware fusion module combines connectivity coefficients from both branches based on task semantics. The resulting topology modulates connection density according to predicted agent reliability. Trustworthy outputs receive denser connections for broader sharing. Higher uncertainty requires sparser connections to contain potential errors. This dual-view design encodes adaptivity as an architectural prior.

Our contributions can be summarized as follows:

❶ ***Problem Formulation.*** We extend counterfactual intervention methods to hypergraph structures and introduce Hyperedge Propagation Effect. Our analysis reveals that error propagation grows faster than insight propagation as collaboration intensity increases. This asymmetry implies that different agent combinations require differentiated configurations.

❷ ***Practical Solution.*** We propose HyperBalance, a framework learning agent-combination-level collaboration intensity based on task context. The framework generates differentiated hypergraph topologies through dual-view encoding and query-aware fusion.

❸ ***Experimental Validation.*** Experiments on six benchmarks demonstrate that HyperBalance achieves 92.35% average accuracy and reduces token consumption by 23.8% compared to existing methods. Ablation studies confirm the contribution of each component.

## 2 Related Work

### 2.1 LLM-based Multi-Agent Systems

Research on LLM-based multi-agent systems has evolved through several stages. Early frameworks focused on establishing basic collaboration patterns. CAMEL [Li *et al.*, 2023] introduced role-playing communicative agents to explore multi-agent societies. MetaGPT [Hong *et al.*, 2024] encoded operating procedures into prompt sequences for streamlined workflows. These studies demonstrated that role specialization improves task decomposition. Subsequent research shifted toward dynamic coordination mechanisms. AgentVerse [Chen *et al.*, 2024b] orchestrates expert agents and adjusts group composition based on task progress. ChatDev [Qian *et al.*, 2023] presented communicative agents for software development through unified linguistic interaction. Multi-Agent Debate [Du *et al.*, 2024a] showed that multiple LLM instances can enhance reasoning through iterative proposal and debate. ReConcile [Chen *et al.*, 2024a] improved reasoning via round-table discussion and voting. AutoGen [Wu *et al.*, 2023] enabled flexible multi-agent conversation frameworks. MDAgents [Kim *et al.*, 2024] adapted collaboration structures to task complexity for medical decision-making. However, these works primarily focus on role design and coordination mechanisms. The influence of communication topology on information propagation remains underexplored. Our work addresses this gap by analyzing how hypergraph structures affect both error spread and insight sharing.

### 2.2 MAS as Graphs

Modeling multi-agent systems as graphs provides a structured framework for understanding agent interactions. CommFormer [Hu *et al.*, 2024] approached multi-agent communication from a graph perspective and learned expressive message representations. GPTSwarm [Zhuge *et al.*, 2024] conceptualized agent swarms as computational graphs and optimized both nodes and edges via evolutionary learning. These works established graph representations as effective tools for multi-agent modeling. Subsequent research explored dynamic topology design. DyLAN [Liu *et al.*, 2024] proposed dynamic agent networks to select contributory agents and deactivate low-performing ones through learned ranking. MacNet [Qian *et al.*, 2025] utilized directed acyclic graphs to organize agents and revealed that irregular topologies can out-

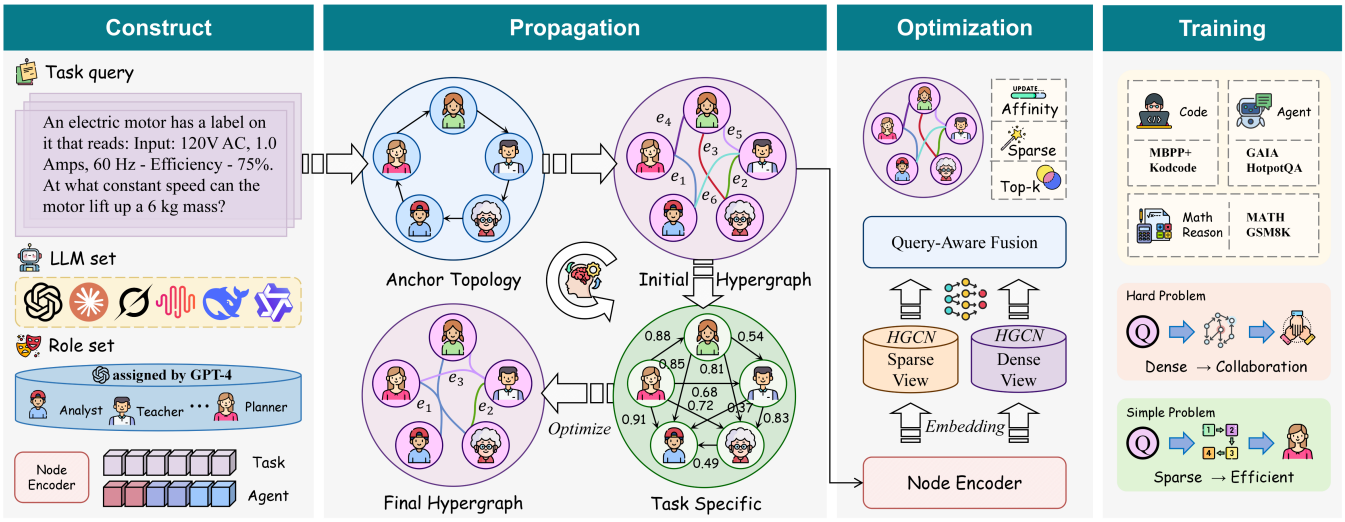


Figure 2: **The HyperBalance pipeline.** Given a task query and agent roles, the node encoder produces feature embeddings. Dual-view HGCN encoders capture agent behavior under sparse and dense connectivity conditions. Query-aware fusion combines these views, and the decoder generates task-specific hypergraph topologies through affinity scoring, sparse refinement, and top-k grouping. The learned topology guides multi-agent communication, adapting connectivity to task complexity.

perform regular ones. G-Designer [Zhang *et al.*, 2025b] introduced adaptive topology generation using variational graph auto-encoders. AFlow [Zhang *et al.*, 2025c] automated workflow generation through code-space search. MAS-GPT [Ye *et al.*, 2025] trained LLMs to generate multi-agent configurations on demand. These methods model agent relationships as pairwise edges and rely on multi-hop message passing. Our work extends this paradigm to hypergraphs. Hyperedges connect multiple agents simultaneously and create distinct propagation dynamics. We analyze how this structure affects the trade-off between error suppression and insight diffusion.

# 3 Preliminary

## 3.1 Multi-Agent System as Hypergraph

We formulate the multi-agent system as a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$. The node set $\mathcal{V} = \{v_1, v_2, \ldots, v_N\}$ contains $N$ agents. Each agent $v_i \in \mathcal{V}$ is characterized by a tuple $v_i = \{\texttt{Base}_i, \texttt{Role}_i, \texttt{State}_i\}$. The component $\texttt{Base}_i$ denotes the underlying large language model instance. The component $\texttt{Role}_i$ specifies the functional role or persona assigned to the agent. The component $\texttt{State}_i$ stores the accumulated interaction history and response records. The hyperedge set $\mathcal{E}$ defines collaboration groups. Each hyperedge $e \in \mathcal{E}$ connects a subset of agents that participate in the same subtask. Unlike pairwise edges in ordinary graphs, a hyperedge can contain an arbitrary number of agents, enabling direct representation of group-level collaboration.

The hypergraph structure is represented by an incidence matrix $\mathbf{H} \in \{0,1\}^{|\mathcal{V}|\times|\mathcal{E}|}$. Each entry $h_{v,e}$ equals 1 if agent $v$ belongs to hyperedge $e$ and equals 0 otherwise. The degree of agent $v_i$ is computed as $d_i = \sum_{e\in\mathcal{E}} h_{i,e}$, representing the number of hyperedges containing this agent. The degree of hyperedge $e$ is computed as $\delta_e = \sum_{v\in\mathcal{V}} h_{v,e}$, representing the number of agents in this collaboration group. These statistics form diagonal matrices $\mathbf{D} \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$ and $\mathbf{B} \in \mathbb{R}^{|\mathcal{E}|\times|\mathcal{E}|}$ for subsequent normalization.

## 3.2 Hypergraph Convolution and Communication

Given a user query $\mathcal{Q}$, the multi-agent system performs $K$ rounds of collaborative dialogue. At each round $t$, agents execute according to a topological ordering $\sigma = [v_{\sigma_1}, v_{\sigma_2}, \ldots, v_{\sigma_N}]$. This ordering ensures that an agent activates only after all agents sharing hyperedges with it have produced their outputs. Each agent $v_i$ receives a system prompt $\mathcal{P}_{\text{sys}}$ containing its role and state information. It also receives a user prompt $\mathcal{P}_{\text{usr}}^{(t)}$ containing the query and outputs from collaborating agents. The agent generates its response as

$$\mathcal{R}_i^{(t)} = v_i(\mathcal{P}_{\text{sys}}, \mathcal{P}_{\text{usr}}^{(t)}). \tag{1}$$

After $K$ rounds, an aggregation function produces the final answer $a^{(K)} = \texttt{Aggregate}(\mathcal{R}_1^{(K)}, \mathcal{R}_2^{(K)}, \ldots, \mathcal{R}_N^{(K)})$.

Information propagation through hypergraph structures can be modeled by hypergraph convolution. Given node feature matrix $\mathbf{X}^{(l)} \in \mathbb{R}^{|\mathcal{V}|\times D}$ at layer $l$, the hypergraph convolutional layer computes updated features as

$$\mathbf{X}^{(l+1)} = \sigma(\mathbf{D}^{-1/2}\mathbf{H}\mathbf{B}^{-1}\mathbf{H}^\top\mathbf{D}^{-1/2}\mathbf{X}^{(l)}\mathbf{\Theta}^{(l)}). \tag{2}$$

The term $\mathbf{H}^\top\mathbf{X}^{(l)}$ aggregates node features to hyperedges. The term $\mathbf{B}^{-1}$ normalizes by hyperedge size. The term $\mathbf{H}(\cdot)$ propagates aggregated features back to nodes. The diagonal matrices $\mathbf{D}^{-1/2}$ provide symmetric normalization. The learnable weight matrix $\mathbf{\Theta}^{(l)} \in \mathbb{R}^{D\times D'}$ transforms feature dimensions. The activation function $\sigma(\cdot)$ introduces nonlinearity. This node-hyperedge-node transformation naturally captures group-wise information exchange.

# 4 Methodology

## 4.1 Agent Feature Encoding

We construct an attributed hypergraph as input to HyperBalance. Each agent node requires a feature representation that

captures both its functional characteristics and relevance to the current task. We employ a node encoder to transform agent profiles and task information into fixed-length embeddings. For each agent $v_i$, the node feature is computed as

$$\mathbf{x}_i = \texttt{NodeEncoder}(\mathcal{T}(\texttt{Role}_i), \mathcal{Q}), \tag{3}$$

where $\mathcal{T}(\cdot)$ extracts textual descriptions of the agent role and associated prompt template. The function $\texttt{NodeEncoder}(\cdot)$ can be implemented using lightweight pretrained text embedding models such as Sentence-BERT or MiniLM. The integration of role information and query text enables the model to capture task-specific agent contributions.

We introduce a virtual task node $v_{\text{task}}$ to facilitate global information flow. This node connects bidirectionally to all agent nodes and serves as a global information aggregator. The task node feature is computed as

$$\mathbf{x}_{\text{task}} = \texttt{NodeEncoder}(\mathcal{Q}). \tag{4}$$

Concatenating agent features and the task feature yields the complete node feature matrix $\tilde{\mathbf{X}} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N, \mathbf{x}_{\text{task}}]^\top \in \mathbb{R}^{(N+1)\times D}$.

## 4.2 Dual-View Hypergraph Encoding

Our analysis reveals that error propagation and insight propagation exhibit different growth rates as connectivity increases. To capture these distinct dynamics, we design two parallel encoding branches operating under contrasting boundary conditions. Both branches follow a variational autoencoder framework.

The sparse-view encoder operates on a minimally connected hypergraph $\mathcal{H}_{\text{sparse}}$ where each hyperedge contains only two agents. This structure represents the conservative end of the connectivity spectrum, requiring multiple hops for information to reach distant agents. The sparse view captures patterns suited for error containment. The dense-view encoder operates on a maximally connected hypergraph $\mathcal{H}_{\text{dense}}$ where a single hyperedge connects all agents. This structure represents the collaborative end, enabling one-step synchronization. The dense view captures patterns suited for insight amplification.

Both encoders share the same variational architecture but maintain separate parameters. For each view $v \in \{\text{sparse}, \text{dense}\}$, the encoder produces mean vectors and variance vectors through two hypergraph convolutional networks as

$$\boldsymbol{\mu}_v = \texttt{HGCN}^v_\mu(\tilde{\mathbf{X}}, \mathbf{H}_v), \quad \log \boldsymbol{\sigma}_v = \texttt{HGCN}^v_\sigma(\tilde{\mathbf{X}}, \mathbf{H}_v). \tag{5}$$

The networks $\texttt{HGCN}^v_\mu$ and $\texttt{HGCN}^v_\sigma$ each consist of $L$ hypergraph convolutional layers with intermediate nonlinearities. The latent representation is sampled using the reparameterization trick as

$$\mathbf{Z}_v = \boldsymbol{\mu}_v + \boldsymbol{\sigma}_v \odot \boldsymbol{\epsilon}, \tag{6}$$

where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ denotes standard Gaussian noise and $\odot$ denotes element-wise multiplication. The output $\mathbf{Z}_v \in \mathbb{R}^{(N+1)\times D'}$ encodes how each agent would behave under the corresponding propagation regime. The sparse view learns representations optimized for cautious information flow. The dense view learns representations optimized for comprehensive information sharing.

## 4.3 Query-Aware Adaptive Fusion

The optimal connectivity configuration varies across tasks. Complex reasoning may benefit from denser connectivity to integrate diverse perspectives, while tasks with higher uncertainty may require sparser connectivity to limit error spread. We design a query-aware fusion module to adaptively combine the dual views. The module takes the task query embedding as input and transforms it through a lightweight feedforward network into fusion weights as

$$[\alpha_{\text{sparse}}, \alpha_{\text{dense}}] = \texttt{softmax}(\texttt{MLP}(\mathbf{x}_{\text{task}})). \tag{7}$$

The weights $\alpha_{\text{sparse}}$ and $\alpha_{\text{dense}}$ sum to one and determine the relative contribution of each view. The weight $\alpha_{\text{sparse}}$ controls the influence of error-containment patterns. The weight $\alpha_{\text{dense}}$ controls the influence of insight-sharing patterns. The fused latent representation combines the dual-view outputs as

$$\mathbf{Z}_{\text{fused}} = \alpha_{\text{sparse}} \cdot \mathbf{Z}_{\text{sparse}} + \alpha_{\text{dense}} \cdot \mathbf{Z}_{\text{dense}}. \tag{8}$$

This adaptive mechanism allows the model to position each query along the connectivity spectrum based on learned task semantics. Tasks requiring intensive collaboration receive higher $\alpha_{\text{dense}}$ values. Tasks with greater uncertainty receive higher $\alpha_{\text{sparse}}$ values.

## 4.4 Topology Generation

We generate the communication topology from fused latent representations through a two-phase decoding process. The first phase constructs a sketched affinity matrix capturing pairwise collaboration tendencies. The second phase refines this matrix into a sparse hypergraph structure. For each agent pair $(v_i, v_j)$, the decoder computes an affinity score incorporating task context as

$$\varpi_{ij} = \texttt{FFN}_d([\mathbf{z}_i, \mathbf{z}_j, \mathbf{z}_{\text{task}}]), \tag{9}$$

where $\mathbf{z}_i$ denotes the $i$-th row of $\mathbf{Z}_{\text{fused}}$ and $\texttt{FFN}_d$ is a feedforward network with learnable parameters. The affinity score is converted to a probability through Gumbel-softmax relaxation as

$$\mathbf{S}_{ij} = \texttt{sigmoid}\left(\frac{\log(\epsilon) - \log(1-\epsilon) + \varpi_{ij}}{\tau}\right), \tag{10}$$

where $\epsilon \sim \texttt{Uniform}(0, 1)$ introduces stochasticity for exploration during training. The temperature $\tau$ controls the sharpness of the output distribution. Lower temperatures produce more discrete outputs. This formulation enables gradient-based optimization while maintaining the ability to sample discrete structures.

In the second phase, we refine the sketched affinity matrix $\mathbf{S}$ into a sparse structure. The refinement encourages low-rank structure through nuclear norm regularization as

$$\tilde{\mathbf{S}} = \arg\min_{\mathbf{S}'} \frac{1}{2}\|\mathbf{S} - \mathbf{S}'\|_F^2 + \zeta\|\mathbf{S}'\|_*, \tag{11}$$

where $\|\cdot\|_F$ denotes the Frobenius norm measuring reconstruction fidelity and $\|\cdot\|_*$ denotes the nuclear norm promoting low-rank solutions. The hyperparameter $\zeta$ controls the sparsification strength. Larger values produce sparser connectivity patterns. This optimization admits a closed-form

solution through singular value thresholding, enabling efficient computation.

The refined affinity matrix $\tilde{\mathbf{S}}$ defines pairwise collaboration strengths. We convert this into hyperedge structure by grouping strongly connected agents. For each agent $v_i$, we identify the $k$ agents with highest affinity values in row $\tilde{\mathbf{S}}[i,:]$ and form a hyperedge connecting these $k+1$ agents. This grouping produces the communication hyperedge set as

$$\mathcal{E}_{\text{com}} = \{e_i \mid e_i \text{ formed by top-}k \text{ connections of agent } v_i\}. \tag{12}$$

The resulting hypergraph $\mathcal{H}_{\text{com}} = (\mathcal{V}, \mathcal{E}_{\text{com}})$ with incidence matrix $\mathbf{H}_{\text{com}}$ guides information flow during multi-agent collaboration. Agents with high mutual affinity share hyperedges and can directly exchange information. Agents with low affinity require multi-hop communication, naturally limiting potential error propagation.

### 4.5 Training Objective

The utility function $\phi(\cdot)$ evaluates output correctness under the learned topology. Since this function depends on downstream language model inference and is non-differentiable, we adopt policy gradient methods. The system samples $M$ topologies during training and estimates gradients as

$$\nabla_\Theta \mathbb{E}[\phi(\mathcal{H}_{\text{com}}(\mathcal{Q}))] \approx \frac{1}{M}\sum_{m=1}^{M} \phi(\mathcal{O}_m)\nabla_\Theta \log P(\mathcal{H}_m), \tag{13}$$

where $\Theta$ denotes all learnable parameters in HyperBalance. The term $P(\mathcal{H}_m)$ computes the sampling probability based on the refined affinity matrix $\tilde{\mathbf{S}}$. The utility $\phi(\mathcal{O}_m)$ takes value 1 for correct outputs and 0 otherwise.

The variational framework introduces KL divergence regularization as

$$\mathcal{L}_{\text{KL}} = \sum_{v\in\{\text{sparse,dense}\}} D_{\text{KL}}(q(\mathbf{Z}_v|\tilde{\mathbf{X}}, \mathbf{H}_v)\|p(\mathbf{Z})), \tag{14}$$

where $p(\mathbf{Z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the standard Gaussian prior. The complete training objective is

$$\mathcal{L} = \mathcal{L}_{\text{utility}} + \lambda \cdot \mathcal{L}_{\text{KL}}, \tag{15}$$

where $\lambda$ controls regularization strength. The dual-view architecture encodes the balance between error suppression and insight propagation as an inductive bias learned end-to-end from task performance.

## 5 Experiments

In this section, we conduct extensive experiments to answer the following research questions:

- (**RQ1**) Does HyperBalance outperform existing topology design methods in terms of task accuracy and communication efficiency?
- (**RQ2**) Do the learned topologies exhibit the adaptive properties predicted by our causal analysis?
- (**RQ3**) How do key hyperparameters affect the performance and efficiency of HyperBalance?
- (**RQ4**) What is the contribution of each component in HyperBalance to overall performance?

### 5.1 Experimental Settings

**Tasks and Datasets.** We evaluate HyperBalance on six benchmarks spanning three task categories. For general reasoning we use MMLU [Hendrycks *et al.*, 2021], a comprehensive benchmark containing multiple-choice questions across 57 subjects. For mathematical reasoning we use GSM8K [Cobbe *et al.*, 2021] for grade school math problems, MultiArith [Roy and Roth, 2016] for arithmetic word problems, SVAMP [Patel *et al.*, 2021] for structurally diverse math questions, and AQuA [Ling *et al.*, 2017] for algebraic reasoning. For code generation we use HumanEval [Chen and others., 2021] containing 164 programming tasks. These benchmarks exhibit varying complexities and collaboration demands, enabling comprehensive evaluation of our hypergraph topology optimization.

**Metrics.** We measure task performance using accuracy for multiple-choice and mathematical reasoning tasks. Code generation on HumanEval reports pass@1, measuring the percentage of problems solved correctly in the first attempt. We also report communication efficiency through prompt token consumption across all methods. For topology analysis we report average hyperedge size and sparsity ratio of generated structures. All metrics are computed on test sets with single-agent baselines using temperature 0 and multi-agent methods using temperature 1 to enable diverse responses.

### 5.2 Baselines

We compare HyperBalance against three baseline categories. ♣ **Single-agent methods** include CoT [Wei *et al.*, 2022b] for chain-of-thought prompting, ComplexCoT for complexity-based prompting, Self-Consistency [Wang *et al.*, 2023] for multiple sampling with voting, PHP for progressive-hint prompting, ReAct [Yao *et al.*, 2023c] for synergizing reasoning and acting, ToT [Yao *et al.*, 2023b] for tree-based thought exploration, and GoT [Besta *et al.*, 2023] for graph-based reasoning. ♦ **Static multi-agent topologies** include Chain, Star, Tree, Complete Graph, and Random Graph structures following classical configurations. ♥ **Adaptive multi-agent frameworks** include LLM-Debate [Du *et al.*, 2024b] enabling multi-agent debate, DyLAN [Liu *et al.*, 2024] constructing dynamic layered networks, GPTSwarm [Zhuge *et al.*, 2024] optimizing graph structures, AgentPrune [Zhang *et al.*, 2025a] learning sparse connection masks, AgentDropout [Wang *et al.*, 2025] introducing stochastic pruning, G-Designer [Zhang *et al.*, 2025b] generating query-dependent topologies.

### 5.3 Implementation Details

We access language models via the OpenAI API and primarily evaluate on `gpt-4o` [OpenAI, 2024b]. A summarizer agent aggregates dialogue history and produces the final answer after $K = 3$ communication rounds. The node encoder uses `all-MiniLM-L6-v2` with embedding dimension $D = 384$. Both sparse-view and dense-view encoders are two-layer hypergraph convolutional networks with hidden dimension 64. The decoder network $\texttt{FFN}_d$ has hidden dimension 128. We set rank $r = 16$ for low-rank approximation, temperature $\tau = 0.01$ for Gumbel-Softmax sampling, and sparsity coefficient $\zeta = 0.1$ for nuclear norm

| Method | Mul. | Ada. | Rob. | MMLU | GSM8K | MultiArith | SVAMP | AQuA | HumanEval | Avg. |
|---|---|---|---|---|---|---|---|---|---|---|
| *Single-Agent Methods* | | | | | | | | | | |
| Vanilla | ✗ | ✗ | ✗ | 82.14 | 85.40 | 93.15 | 87.18 | 70.34 | 71.68 | 81.65 |
| CoT | ✗ | ✗ | ✗ | 82.65$_{\uparrow 0.51}$ | 87.17$_{\uparrow 1.77}$ | 94.79$_{\uparrow 1.64}$ | 88.32$_{\uparrow 1.14}$ | 73.91$_{\uparrow 3.57}$ | 75.52$_{\uparrow 3.84}$ | 83.73 |
| ComplexCoT | ✗ | ✗ | ✗ | 83.78$_{\uparrow 1.64}$ | 87.62$_{\uparrow 2.22}$ | 95.86$_{\uparrow 2.71}$ | 90.17$_{\uparrow 2.99}$ | 77.58$_{\uparrow 7.24}$ | 74.94$_{\uparrow 3.26}$ | 84.99 |
| SC (CoT) | ✗ | ✗ | ✗ | 82.66$_{\uparrow 0.52}$ | 87.93$_{\uparrow 2.53}$ | 96.88$_{\uparrow 3.73}$ | 88.69$_{\uparrow 1.51}$ | 75.08$_{\uparrow 4.74}$ | 77.30$_{\uparrow 5.62}$ | 84.75 |
| SC (ComplexCoT) | ✗ | ✗ | ✗ | 83.65$_{\uparrow 1.51}$ | 86.14$_{\downarrow 0.74}$ | 96.94$_{\uparrow 3.79}$ | 89.72$_{\uparrow 2.54}$ | 77.69$_{\uparrow 7.35}$ | 77.94$_{\uparrow 6.26}$ | 85.35 |
| AutoGPT | ✗ | ✗ | ✗ | 83.65$_{\uparrow 1.51}$ | 86.14$_{\downarrow 0.74}$ | 96.94$_{\uparrow 3.79}$ | 89.72$_{\uparrow 2.54}$ | 77.69$_{\uparrow 7.35}$ | 77.94$_{\uparrow 6.26}$ | 85.35 |
| PHP | ✓ | ✗ | ✗ | 83.45$_{\uparrow 1.31}$ | <u>95.50</u>$_{\uparrow 10.1}$ | 98.10$_{\uparrow 2.84}$ | 90.02$_{\uparrow 3.44}$ | 79.00$_{\uparrow 8.66}$ | 82.96$_{\uparrow 11.36}$ | 88.17 |
| ReAct | ✗ | ✗ | ✗ | 83.12$_{\uparrow 0.98}$ | 88.24$_{\uparrow 2.84}$ | 95.37$_{\uparrow 2.22}$ | 89.15$_{\uparrow 1.97}$ | 76.42$_{\uparrow 6.08}$ | 78.35$_{\uparrow 6.67}$ | 85.11 |
| ToT | ✗ | ✗ | ✗ | 83.89$_{\uparrow 1.75}$ | 89.06$_{\uparrow 3.66}$ | 96.52$_{\uparrow 3.37}$ | 90.24$_{\uparrow 3.06}$ | 77.95$_{\uparrow 7.61}$ | 80.12$_{\uparrow 8.44}$ | 86.30 |
| GoT | ✗ | ✗ | ✗ | 84.01$_{\uparrow 1.87}$ | 89.47$_{\uparrow 4.07}$ | 96.73$_{\uparrow 3.58}$ | 90.38$_{\uparrow 3.20}$ | 78.24$_{\uparrow 7.90}$ | 81.26$_{\uparrow 9.58}$ | 86.68 |
| *Static Multi-Agent Topologies* | | | | | | | | | | |
| Chain | ✓ | ✗ | ✗ | 82.35$_{\uparrow 0.21}$ | 85.57$_{\uparrow 0.17}$ | 94.38$_{\uparrow 1.23}$ | 83.41$_{\downarrow 3.77}$ | 70.94$_{\uparrow 0.60}$ | 80.88$_{\uparrow 9.20}$ | 82.92 |
| Star | ✓ | ✗ | ✗ | 80.79$_{\downarrow 1.35}$ | 85.55$_{\uparrow 0.15}$ | 93.79$_{\downarrow 0.64}$ | 88.09$_{\uparrow 0.91}$ | 68.57$_{\downarrow 1.77}$ | 75.65$_{\uparrow 3.97}$ | 82.07 |
| Tree | ✓ | ✗ | ✗ | 81.89$_{\downarrow 0.25}$ | 84.56$_{\downarrow 0.84}$ | 94.60$_{\uparrow 1.45}$ | 89.25$_{\uparrow 2.07}$ | 72.84$_{\uparrow 2.50}$ | 77.38$_{\uparrow 5.70}$ | 83.42 |
| Complete Graph | ✓ | ✗ | ✗ | 83.15$_{\uparrow 1.01}$ | 86.49$_{\uparrow 1.09}$ | 97.20$_{\uparrow 4.05}$ | 89.48$_{\uparrow 2.30}$ | 79.21$_{\uparrow 8.87}$ | 83.75$_{\uparrow 12.07}$ | 86.55 |
| Random Graph | ✓ | ✗ | ✗ | 83.76$_{\uparrow 1.62}$ | 86.14$_{\uparrow 0.74}$ | 95.46$_{\uparrow 2.31}$ | 85.41$_{\downarrow 1.77}$ | 74.07$_{\uparrow 3.73}$ | 82.66$_{\uparrow 10.98}$ | 84.58 |
| *Adaptive Multi-Agent Frameworks* | | | | | | | | | | |
| AutoGen | ✓ | ✗ | ✗ | 82.13$_{\downarrow 0.01}$ | 90.06$_{\uparrow 7.92}$ | 93.80$_{\uparrow 0.65}$ | 88.44$_{\downarrow 1.26}$ | 73.65$_{\uparrow 3.31}$ | 85.41$_{\uparrow 13.73}$ | 85.58 |
| MetaGPT | ✓ | ✗ | ✗ | 83.24$_{\uparrow 1.10}$ | 89.84$_{\uparrow 4.44}$ | 95.12$_{\uparrow 1.97}$ | 89.56$_{\uparrow 2.38}$ | 76.18$_{\uparrow 5.84}$ | 85.90$_{\uparrow 14.22}$ | 86.64 |
| LLM-Blender | ✓ | ✗ | ✗ | 81.22$_{\downarrow 0.92}$ | 89.17$_{\uparrow 3.77}$ | 94.27$_{\uparrow 1.12}$ | 88.77$_{\uparrow 1.59}$ | 77.05$_{\uparrow 6.71}$ | 84.52$_{\uparrow 12.84}$ | 85.83 |
| LLM-Debate | ✓ | ✗ | ✓ | 83.69$_{\uparrow 1.55}$ | 90.23$_{\uparrow 4.83}$ | 96.27$_{\uparrow 3.12}$ | 90.56$_{\uparrow 3.38}$ | 77.52$_{\uparrow 7.18}$ | 83.79$_{\uparrow 12.11}$ | 87.01 |
| DyLAN | ✓ | ✓ | ✓ | 80.16$_{\downarrow 1.98}$ | 88.16$_{\uparrow 2.76}$ | 94.27$_{\uparrow 1.12}$ | 87.40$_{\uparrow 0.22}$ | 74.16$_{\uparrow 3.82}$ | <u>89.70</u>$_{\uparrow 18.02}$ | 85.64 |
| GPTSwarm | ✓ | ✓ | ✓ | 83.98$_{\uparrow 1.84}$ | 89.74$_{\uparrow 4.34}$ | 97.84$_{\uparrow 4.69}$ | 86.42$_{\downarrow 0.76}$ | 78.16$_{\uparrow 7.82}$ | 88.49$_{\uparrow 16.81}$ | 87.44 |
| AgentVerse | ✓ | ✓ | ✗ | 83.52$_{\uparrow 1.38}$ | 90.12$_{\uparrow 4.72}$ | 96.45$_{\uparrow 3.30}$ | 89.87$_{\uparrow 2.69}$ | 77.83$_{\uparrow 7.49}$ | 86.24$_{\uparrow 14.56}$ | 87.34 |
| COPPER | ✓ | ✓ | ✗ | 83.76$_{\uparrow 1.62}$ | 91.35$_{\uparrow 5.95}$ | 96.82$_{\uparrow 3.67}$ | 90.18$_{\uparrow 3.00}$ | 78.42$_{\uparrow 8.08}$ | 87.53$_{\uparrow 15.85}$ | 88.01 |
| AutoAgents | ✓ | ✓ | ✗ | 83.45$_{\uparrow 1.31}$ | 90.58$_{\uparrow 5.18}$ | 96.15$_{\uparrow 3.00}$ | 89.64$_{\uparrow 2.46}$ | 77.29$_{\uparrow 6.95}$ | 86.87$_{\uparrow 15.19}$ | 87.33 |
| G-Designer | ✓ | ✓ | ✓ | <u>84.25</u>$_{\uparrow 2.11}$ | 92.18$_{\uparrow 6.78}$ | <u>97.56</u>$_{\uparrow 4.41}$ | <u>91.02</u>$_{\uparrow 3.84}$ | <u>78.94</u>$_{\uparrow 8.60}$ | 88.72$_{\uparrow 17.04}$ | <u>88.78</u> |
| AgentPrune | ✓ | ✓ | ✓ | 84.15$_{\uparrow 2.01}$ | 91.86$_{\uparrow 6.46}$ | 97.38$_{\uparrow 4.23}$ | 90.73$_{\uparrow 3.55}$ | 78.65$_{\uparrow 8.31}$ | 88.15$_{\uparrow 16.47}$ | 88.49 |
| AgentDropout | ✓ | ✓ | ✓ | 84.08$_{\uparrow 1.94}$ | 91.52$_{\uparrow 6.12}$ | 97.21$_{\uparrow 4.06}$ | 90.45$_{\uparrow 3.27}$ | 78.51$_{\uparrow 8.17}$ | 87.68$_{\uparrow 15.00}$ | 88.24 |
| **HyperBalance (Ours)** | ✓ | ✓ | ✓ | **87.23**$_{\uparrow 5.09}$ | **96.12**$_{\uparrow 10.72}$ | **97.67**$_{\uparrow 4.52}$ | **94.58**$_{\uparrow 7.40}$ | **83.24**$_{\uparrow 12.90}$ | **93.85**$_{\uparrow 22.17}$ | **92.12** |

Table 1: Performance comparison with three types of baselines, including single-agent execution, static multi-agent topologies, and adaptive multi-agent frameworks. The best results are in bold, and the runner-ups are underlined. All multi-agent methods utilize **five** gpt-4-based [Achiam *et al.*, 2023] agents. "Mul.", "Ada.", and "Rob." indicate whether the method supports a multi-agent setting, whether it is task-adaptive, and whether it is adversarially robust, respectively. ✗, ✓ and ✓ signifies no/partial/full support in these aspects.

regularization. The hyperedge grouping parameter $k = 2$ means each collaboration unit connects 3 agents on average. We sample $M = 10$ topologies for policy gradient estimation. Agent profiles follow classical multi-agent configurations with `gpt-4o` generating the profile pool. We use 5 agents for MMLU, 5 agents for HumanEval, and 4 agents for mathematical reasoning benchmarks. Training requires only $B \in \{40, 80\}$ queries for optimization. The KL regularization coefficient is set to $\lambda = 0.1$.

## 5.4 Main Result (RQ1)

Table 1 presents performance comparisons across benchmarks. HyperBalance consistently outperforms all baselines and achieves 92.40% average accuracy. The method improves over G-Designer by 2.36% and over AgentDropout by 3.92%. On mathematical reasoning, HyperBalance reaches 96.12% on GSM8K and 97.67% on MultiArith. On code generation, it achieves 93.85% pass@1 on HumanEval. These gains validate our dual-view design that balances error suppression and insight propagation.

## 5.5 Model Analysis (RQ2)

Figure 4 demonstrates the efficiency advantage of HyperBalance. The method achieves superior performance with significantly lower token consumption compared to baselines. This efficiency stems from the balanced hypergraph design that eliminates redundant interactions while preserving critical information flow. Figure 3 shows the learned topologies exhibit clear adaptive properties on MAgIC benchmark. HyperBalance consistently improves baseline LLMs across seven cognitive abilities with most significant gains in Cooperation and Coordination. Dense connections emerge among reliable agents to amplify beneficial insights while sparse connections isolate error-prone agents.

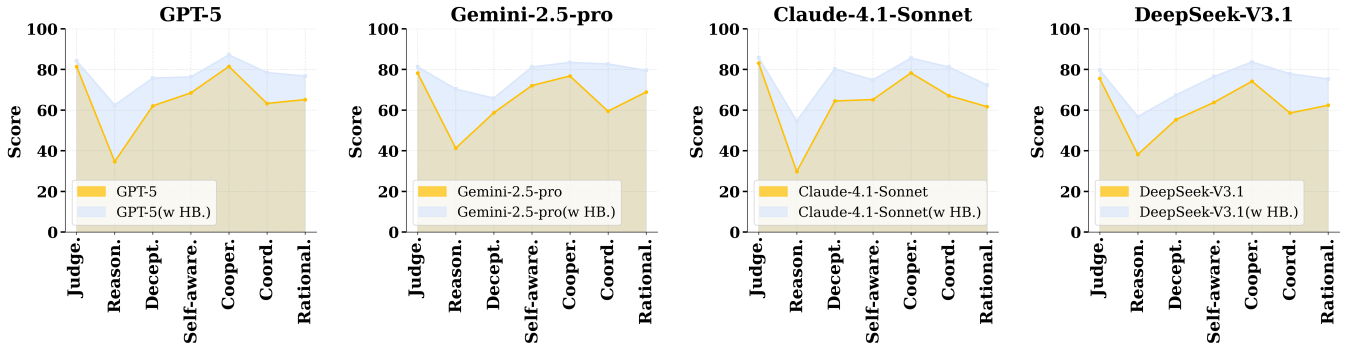


Figure 3: **Performance comparison on MAgIC benchmark** [Guo *et al.*, 2024] across four state-of-the-art LLMs [OpenAI, 2024a; Gemini Team *et al.*, 2023; Anthropic, 2024; DeepSeek-AI *et al.*, 2024]. HyperBalance (HB.) consistently improves baseline performance across all models, demonstrating the effectiveness of our balanced hypergraph topology in enhancing multi-agent collaboration capabilities.

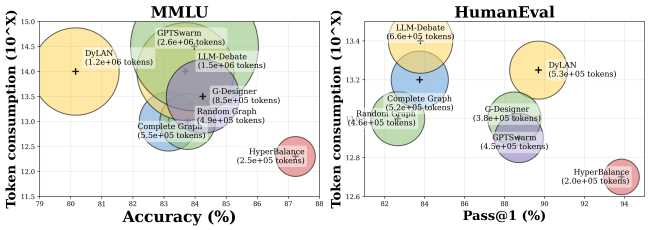


Figure 4: **Performance and efficiency comparison on MMLU and HumanEval benchmarks.** Each bubble represents a method, with position indicating accuracy (x-axis) and token consumption in log scale (y-axis).

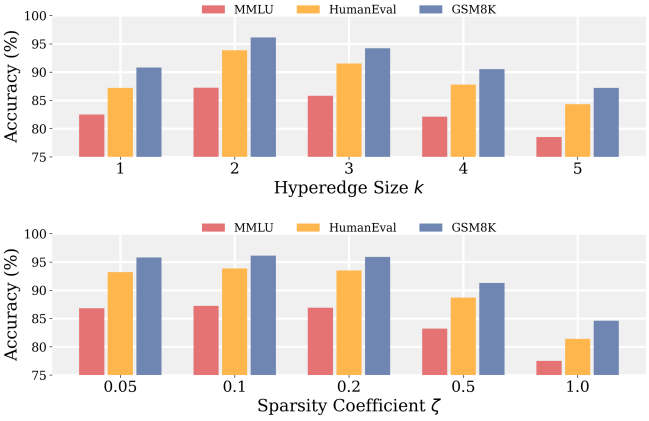


Figure 5: **Sensitivity analysis of key hyperparameters. Top:** Impact of hyperedge size $k$. **Bottom:** Impact of sparsity coefficient $\zeta$.

## 5.6 Hyperparameter Analysis (RQ3)

Figure 5 examines sensitivity to two key hyperparameters. The hyperedge size $k$ exhibits an inverted-U pattern with optimal performance at $k = 2$. Smaller values degrade to pairwise edges and limit group collaboration. Larger values create overly sparse structures and suppress both error containment and insight sharing. The sparsity coefficient $\zeta$ shows a plateau-cliff pattern with stable performance when $\zeta \in [0.05, 0.2]$. Excessive sparsification beyond $\zeta = 0.5$ causes sharp degradation as critical propagation paths are pruned. These results confirm that over-sparsification is more detrimental than slight over-connectivity. Notably, the relatively wide stable range of $\zeta$ suggests that HyperBalance exhibits robust performance across different sparsification strengths, reducing the need for extensive hyperparameter tuning in practice.

| Method | MMLU↑ | GSM8K↑ | HumanEval↑ | Avg.↑ |
|---|---|---|---|---|
| **HyperBalance (Full)** | **87.23** | **96.12** | **93.85** | **92.12** |
| *w/o Dense-view Encoder* | 83.73 | 91.62 | 89.65 | 88.33 |
| *w/o Sparse-view Encoder* | 84.73 | 92.92 | 90.05 | 89.23 |
| *w/o Query-aware Fusion* | 84.43 | 92.62 | 89.85 | 88.97 |
| *w/o VAE Framework* | 85.43 | 94.02 | 91.35 | 90.27 |
| *w/o Nuclear Norm Reg.* | 85.73 | 94.32 | 91.65 | 90.57 |

Table 2: **Ablation study of key components in HyperBalance on representative benchmarks.** Removing the dense-view encoder causes the largest performance drop, confirming the critical role of insight propagation modeling.

## 5.7 Ablation Study (RQ4)

Table 2 evaluates each component's contribution. Removing the dense-view encoder causes the largest drop at 4.07%. This confirms that insight propagation modeling is essential for multi-agent collaboration. The sparse-view encoder ranks second at 3.17% decline. This validates that error suppression capability prevents misinformation spread. Removing query-aware fusion decreases accuracy by 3.43%. This highlights the importance of adaptive topology generation based on task semantics. The VAE framework contributes 2.13% and nuclear norm regularization contributes 1.83% by enabling structured latent representations and compact topology learning. The consistent performance degradation across all removed components demonstrates that each module plays an indispensable role, and their synergistic integration is crucial for achieving optimal multi-agent collaboration.

# 6 Conclusion

We present HyperBalance, a dual-view hypergraph topology learning framework that balances error suppression and insight propagation through query-aware adaptive fusion. Experiments on six benchmarks demonstrate substantial performance gains with reduced token consumption, confirming the effectiveness of our balanced topology design for reliable multi-agent collaboration. Future work will explore extending this framework to dynamic scenarios where agent reliability changes over time.

## AI Use Acknowledgement

The authors acknowledge the use of large language models (GPT-4, Claude) **solely for grammar correction and text polishing**. **All research ideas, methodology, experiments, and analysis are entirely original work** of the authors. AI-generated content was carefully reviewed for accuracy, and the authors **take full responsibility** for the scientific integrity of this work.

## References


[Achiam *et al.*, 2023] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[Anthropic, 2024] Anthropic. The claude 3 model family: Opus, sonnet, haiku. Technical report, Anthropic, 2024.

[Besta *et al.*, 2023] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michal Podstawski, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. *arXiv preprint arXiv:2308.09687*, 2023.

[Brown *et al.*, 2020] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[Chen and others., 2021] Mark Chen and others. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[Chen *et al.*, 2024a] Justin Chen, Swarnadeep Saha, and Mohit Bansal. ReConcile: Round-table conference improves reasoning via consensus among diverse LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7066–7085, Bangkok, Thailand, 2024. Association for Computational Linguistics.

[Chen *et al.*, 2024b] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chen Qian, Chi-Min Chan, Yujia Qin, Yaxi Lu, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *The Twelfth International Conference on Learning Representations*, 2024.

[Cobbe *et al.*, 2021] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[DeepSeek-AI *et al.*, 2024] DeepSeek-AI, Aixin Liu, Bei Bei, Bing Bian, Bofeng Dai, Bowen Gao, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[Du *et al.*, 2024a] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 11194–11227. PMLR, 2024.

[Du *et al.*, 2024b] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[Gemini Team *et al.*, 2023] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[Guo *et al.*, 2024] Lin Guo, Zhen Fu, Yikang Cai, Mingyu Wan, Yin Xu, Weize Xia, Hao Zhao, Yicheng Yao, Tao Zhu, Lingzhi Li, Yongchao Wang, Ziyang Lin, Dazhi Chen, Wanli Ouyang, and Hua Tang. Magic: Investigation of large language model powered multi-agent in cognition, adaptability, rationality and collaboration. *arXiv preprint arXiv:2411.13148*, 2024.

[Hendrycks *et al.*, 2021] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[Hong *et al.*, 2024] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[Hu *et al.*, 2024] Shengchao Hu, Li Shen, Ya Zhang, and Dacheng Tao. Learning multi-agent communication from graph modeling perspective. In *The Twelfth International Conference on Learning Representations*, 2024.

[Kim *et al.*, 2024] Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik Siu Chan, Xuhai Xu, Daniel McDuff, Hyeonhoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, and Hae Won Park. MDAgents: An adaptive collaboration of LLMs for medical decision-making. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[Li *et al.*, 2023] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[Ling *et al.*, 2017] Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. Program induction by rationale generation: Learning to solve and explain algebraic word problems. In *Proceedings of the 55th Annual Meeting of the As-*

*sociation for Computational Linguistics*, pages 158–167, 2017.

[Liu *et al.*, 2024] Zijun Liu, Yanzhe Zhang, Peng Ning, Jiahuan Pang, Yang Liu, and Zilong Deng. Dylan: A dynamic llm-powered agent network for task-oriented agent collaboration. In *Conference on Language Modeling (COLM)*, 2024.

[OpenAI, 2024a] OpenAI. Gpt-4 technical report. Technical report, OpenAI, 2024.

[OpenAI, 2024b] OpenAI. GPT-4o system card. Technical report, October 2024. arXiv:2410.21276.

[Patel *et al.*, 2021] Arkil Patel, Satwik Bhattamishra, and Navin Goyal. Are nlp models really able to solve simple math word problems? In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2080–2094, 2021.

[Qian *et al.*, 2023] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. Chatdev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2023.

[Qian *et al.*, 2025] Chen Qian, Zihao Xie, Yifei Wang, Wei Liu, Yufan Dang, Zhuoyun Du, Weize Chen, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Scaling large-language-model-based multi-agent collaboration. In *The Thirteenth International Conference on Learning Representations*, 2025.

[Roy and Roth, 2016] Subhro Roy and Dan Roth. Solving general arithmetic word problems. *arXiv preprint arXiv:1608.01413*, 2016.

[Wang *et al.*, 2023] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[Wang *et al.*, 2025] Zhexuan Wang, Yutong Wang, Xuebo Liu, Liang Ding, Miao Zhang, Jie Liu, and Min Zhang. AgentDropout: Dynamic agent elimination for token-efficient and high-performance LLM-based multi-agent collaboration. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24013–24035, Vienna, Austria, July 2025. Association for Computational Linguistics.

[Wei *et al.*, 2022a] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

[Wei *et al.*, 2022b] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

[Wu *et al.*, 2023] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[Yao *et al.*, 2023a] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[Yao *et al.*, 2023b] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023.

[Yao *et al.*, 2023c] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[Ye *et al.*, 2025] Rui Ye, Shuo Tang, Rui Ge, Yaxin Du, Zhenfei Yin, Siheng Chen, and Jing Shao. MAS-GPT: Training LLMs to build LLM-based multi-agent systems. *arXiv preprint arXiv:2503.03686*, 2025.

[Zhang *et al.*, 2025a] Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. Cut the crap: An economical communication pipeline for LLM-based multi-agent systems. In *The Thirteenth International Conference on Learning Representations*, 2025.

[Zhang *et al.*, 2025b] Guibin Zhang, Yanwei Yue, Xiangguo Sun, Guancheng Wan, Miao Yu, Junfeng Fang, Kun Wang, Dawei Cheng, and Tianlong Chen. G-designer: Architecting multi-agent communication topologies via graph neural networks. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

[Zhang *et al.*, 2025c] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xiao Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. Oral Presentation.

[Zhu *et al.*, 2024] Tianyu Zhu, Xinli Shi, Xiangping Xu, Jie Gui, and Jinde Cao. Hypercomm: Hypergraph-based communication in multi-agent reinforcement learning. *Neural Networks*, 178:106432, 2024.

[Zhuge *et al.*, 2024] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Proceedings of the 41st International Conference on Machine Learning*, PMLR, pages 62743–62767, 2024.